

FPGA hardware linear regression implementation using fixed-point arithmetic

Willian de Assis Pedrobon

Ferreira

Scholl of Engineering

São Paulo State University (UNESP)

Brazil, Ilha Solteira

willian.ferreira@unesp.br

Ian Grout

Department of Electronic and

Computer Engineering

University of Limerick

Ireland, Limerick

Ian.Grout@ul.ie

Alexandre César Rodrigues da

Silva

Scholl of Engineering

São Paulo State University (UNESP)

Brazil, Ilha Solteira

alexandre.cr.silva@unesp.br

ABSTRACT

In this paper, a hardware design based on the field programmable gate array (FPGA) to implement a linear regression algorithm is presented. The arithmetic operations were optimized by applying a fixed-point number representation for all hardware based computations. A floating-point number training data point was initially created and stored in a personal computer (PC) which was then converted to fixed-point representation and transmitted to the FPGA via a serial communication link. With the proposed VHDL design description synthesized and implemented within the FPGA, the custom hardware architecture performs the linear regression algorithm based on matrix algebra considering a fixed size training data point set. To validate the hardware fixed-point arithmetic operations, the same algorithm was implemented in the Python language and the results of the two computation approaches were compared. The power consumption of the proposed embedded FPGA system was estimated to be 136.82 mW.

CCS CONCEPTS

• **Hardware** → **Reconfigurable logic and FPGAs**; *Digital signal processing*; • **Computing methodologies** → *Machine learning*.

KEYWORDS

Machine learning, linear regression, hardware, fixed-point arithmetic, FPGA

ACM Reference Format:

Willian de Assis Pedrobon Ferreira, Ian Grout, and Alexandre César Rodrigues da Silva. 2019. FPGA hardware linear regression implementation using fixed-point arithmetic. In *32nd Symposium on Integrated Circuits and Systems Design (SBCCI '19)*, August 26–30, 2019, Sao Paulo, Brazil. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3338852.3339853>

1 INTRODUCTION

Machine learning has recently become one of the most interesting areas in the field of artificial intelligence. This success is due to the

possibility of developing efficient classification systems supported by learning algorithms, which extract information from monitored situations and enable the prediction of new events. These algorithms have increased the cognitive ability of computational interfaces and improved the effectiveness of autonomous services in a range of applications.

One of the machine learning application fields is the Internet of things (IoT), where the purpose is to create networks of embedded computing devices to provide human-machine interaction through data sharing. With the implementation of learning algorithms, electronic devices are able to adapt themselves according to existing behaviors and to assist human in daily activities.

Connecting IoT and machine learning brings new challenges for intelligent embedded systems. The complexity of information processing has increased due to the high number of interconnected devices. Large IoT databases can be used to store IoT sourced data and the streaming of sampled data can happen in short periods of time. Dense data issues influence machine learning algorithm implementation, principally to development IoT systems incorporate algorithms running on the IoT computing device. It is necessary to manage the data storage in order to efficiently perform the training algorithm that must operate in a required processing time. This is a research area with high demand [12].

Currently, there are several digital devices to implement embedded systems, which can be more appropriate according to the application. A comparison between the performance and flexibility of digital devices presented in [1] is shown in Figure 1.

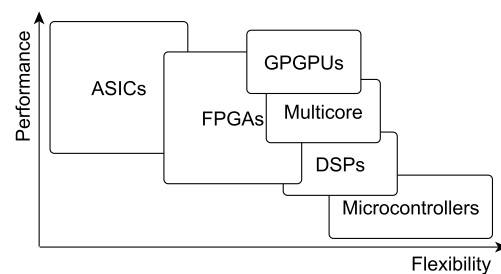


Figure 1: Balance between performance and flexibility of digital devices.

Due to the performance demands of machine learning algorithms, embedded microcontroller based processing can suffer from bottlenecks in data processing actions. The use of application specific

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SBCCI '19, August 26–30, 2019, Sao Paulo, Brazil

© 2019 Association for Computing Machinery.

ACM ISBN 978-1-4503-6844-5/19/08...\$15.00

<https://doi.org/10.1145/3338852.3339853>

integrated circuits (ASICs) can fulfill the performance requirements, however, its fixed architecture and high implementation cost make them less feasible to implement custom machine learning systems. As an alternative, the field programmable gate array (FPGA) becomes the most balanced option between performance and flexibility, being a viable alternative for implementing embedded machine learning systems.

Some of the main FPGA features that allow the optimization of machine learning algorithms are the reprogrammable architecture, high performance, parallelism, high data throughput, and real-time processing [13, 16].

With the high number of applications that require efficient data processing, machine learning has the potential to become the next major hardware development focus [5].

To accelerate embedded machine learning algorithm several researchers explore digital signal processor (DSP) concepts in FPGAs to improve arithmetic computation. To increase productivity in this field, some FPGA vendors such as Altera [7], Xilinx [19] and Lattice Semiconductor [3] provide tools that support the implementation of synthesizable DSP hardware.

One of the machine learning approaches that FPGA are applied in order to implement custom design is the simple linear regression [6]. There is also interest to study FPGAs features in other regression models, such as multiple linear regression [4].

In order to obtain better hardware efficiency, it is possible to apply quantization in the DPS operations, which objective is to work with less data rate through decreasing the number precision. Even with the limited value representation, the quantization has been guaranteed good results in machine learning applications in both floating-point arithmetic [9] and fixed-point [10].

Considering the presented context, in this article is proposed the implementation of an FPGA based embedded system to implement a linear regression algorithm in hardware. The objective of this learning algorithm is to obtain the best straight line to interpolate training data points. The custom architecture was developed using the Altera DE2-115 development board and the VHDL hardware description language. To perform the arithmetic operations, the floating-point input training data set was initially converted to a fixed-point representation. The embedded multipliers within the FPGA were used in this paper allowed to parallelize the linear regression computation and prediction algorithms, allowing rapidly to implement a prototype system that had proved the feasibility of the proposed custom architecture.

A personal computer (PC) was used in the system evaluation. A Python script converted the training data to fixed-point arithmetic and transmitted the data to the FPGA via a serial communication link. The results of hardware linear regression and predictions were transmitted from the FPGA back to the PC for evaluation. To validate the fixed-point arithmetic modeling, the same algorithm was described in hardware and coded in Python, whose arithmetic operations were performed using floating-point double precision. The results and processing performance validated the proposed custom linear regression architecture.

The improved arithmetic operations optimization archived with the fixed-point representation in the proposed custom hardware might be used within a larger FPGA based design to perform real-time predictions with a continuous updating of the linear regression

parameters. Therefore the system might give a real-time and embedded analysis of the data that has just been sampled.

This paper is organized as follows. In Section 2 the linear regression algorithm and the hardware design are described. In Section 3 the system evaluation and analysis are presented, and in Section 4 the conclusions are exposed.

2 SIMPLE LINEAR REGRESSION

The purpose of regression is to model a function to predict an output value from input variables. One of the basic techniques in this field is the linear regression, which is based on the solution of linear systems by processing data organized in linear algebra structures such as matrices and vectors.

To obtain results closer to those desired, the parameters of the prediction function (θ vector) are adjusted using training examples and optimization algorithms. This is the basic objective of machine learning techniques, which aim to minimize the error of a prediction function to obtain an efficient and autonomous classification system [15].

In this paper a simple linear regression was implemented, that is, the y prediction function receives only one x input parameter, as equation 1 shows:

$$\begin{aligned} y &= ax + b \\ &= \theta_1 x + \theta_0 \end{aligned} \quad (1)$$

The adjustment of θ values was carried out applying the least-squares estimation method [17] via matrix algebra operations:

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix} = (X^T X)^{-1} X^T y \quad (2)$$

where X and y were the training data points.

Although the development of equation 2 was implemented in hardware to accelerate the linear regression algorithm, in addition to the FPGA a computer was also used for management and processing the training data point. The flowchart of the software and hardware operations is shown in Figure 2.

A Python script read the training examples stored in computer, that is, the X and y data structures. Before transmitting the information to the FPGA via serial communication, the data were converted to fixed-point arithmetic. In hardware, the simple linear regression algorithm and prediction considering the X values were performed. The θ values and prediction results were then transmitted back to the computer and converted to floating-point double precision numbers.

The proposed methodology to achieve the custom simple linear regression hardware architecture is presented below.

2.1 Hardware design

The simple linear regression algorithm was implemented using the VHDL hardware description language and the targeted the Altera DE2-115 development board and the Altera Cyclone IV EP4CE115F29C8 FPGA.

An important issue to consider in order to obtain synthesizable hardware description is the arithmetic operations. Aiming the optimization of execution time, the fixed-point arithmetic was selected. Decimal values are modeled as signed integer numbers using 32-bit

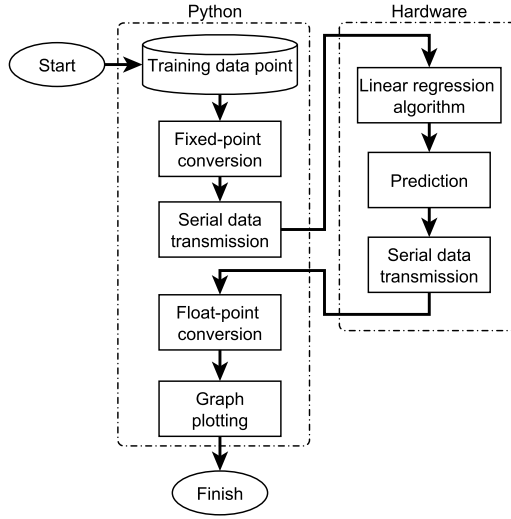


Figure 2: Flowchart of the proposed simple linear regression system.

STD_LOGIC_VECTOR arrays. With this approach, the most significant bit is the sign bit, and the remaining bits constitute an integer and a fractional component. In this paper, the number of bits to represent the fractional and integer parts of the fixed-point numbers is respectively 16 and 15 bits as presented in Figure 3.

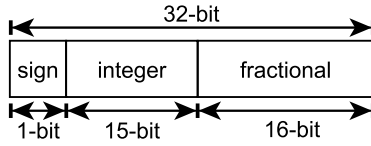


Figure 3: Fixed-point number representation applied in this paper.

Training data stored in a computer were converted to 32-bit fixed-point representation applying the Kibo Python library [14] to be transmitted to the FPGA. When the Python script received the hardware results, the data were converted back to floating-point arithmetic.

To solve the equation 2, the linear algebra operations were split into sequential operations coded in a PROCESS VHDL structure with a start signal in sensitive list. The arithmetic operation data flow was analyzed to optimize the hardware design. The modeling obtained is presented in Algorithm 1, which generated a custom intellectual property (IP) entity presented in Figure 4(a).

As the input to the simple linear regression algorithm received the sampled training data point stored in X and y matrix as follows:

$$X = \begin{bmatrix} 1 & x_0 \\ 1 & x_1 \\ \vdots & \vdots \\ 1 & x_n \end{bmatrix}, \quad y = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{bmatrix} \quad (3)$$

Algorithm 1: Linear regression via matrix algebra

Input: Training data point ($X = [x_0 \ \dots \ x_{n-1}]$ and $y = [y_0 \ \dots \ y_{n-1}]$) and start signal

Output: $\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix}$

```

1  n: number of training examples ;
2  if rising_edge(start) then
3      Compute  $(X^T X) = \begin{bmatrix} n & \sum_{i=0}^{n-1} x_i \\ \sum_{i=0}^{n-1} x_i & \sum_{i=0}^{n-1} x_i^2 \end{bmatrix} = \begin{bmatrix} n & a \\ a & b \end{bmatrix}$  ;
4      Compute  $(X^T X)^{-1} = \frac{1}{nb-a^2} \begin{bmatrix} b & -a \\ -a & n \end{bmatrix} = \begin{bmatrix} c & d \\ d & e \end{bmatrix}$  ;
5      Compute  $(X^T X)^{-1} X^T = \begin{bmatrix} c+dx_0 & \dots & c+dx_{n-1} \\ d+ex_0 & \dots & d+ex_{n-1} \end{bmatrix} = \begin{bmatrix} t_{00} & \dots & t_{0n-1} \\ t_{10} & \dots & t_{1n-1} \end{bmatrix}$  ;
6      Compute  $(X^T X)^{-1} X^T y = \begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix} = \begin{bmatrix} \sum_{i=0}^{n-1} t_{0i} y_i \\ \sum_{i=0}^{n-1} t_{1i} y_i \end{bmatrix}$  ;
7  end
8  return  $\theta$ 

```

If the traditional linear algebra matrix multiplication was considered to solve $(X^T X)$, unnecessary multiplications would be performed due to the first column of 1's in matrix X . In order to optimize the number of arithmetic operations, the linear regression VHDL code received only the second column of X as an array data structure. The development of $(X^T X)$ is presented in line 3 of Algorithm 1.

To compute the inverse of $(X^T X)$, a 2x2 matrix inversion technique shown in line 4 was applied. The reciprocal of the determinant $D = nb - a^2$ was calculated applying the iterative Newton-Raphson approximation algorithm:

$$x_{i+1} = x_i(2 - Dx_i) \quad (4)$$

It was necessary to determinate an initial x_0 approximation value, which could be obtained from a look-up table [2]. It was also possible apply an approximation polynomial [8]. In this paper, the x_0 was determined as being the next power two base value greater than the absolute of D . This technique does not consume additional memory for storing a look-up table and avoid arithmetic operations necessary to solve the approximation polynomial. The number of iterations to solve the equation 4 was set to 3.

In line 5, the multiplication of matrix $(X^T X)^{-1}$ and X^T does not use the conventional algebra technique due to the 1's line of X^T , which each line of its matrix result is liner combined with y input array in line 6 to result the θ output.

With the finalization of the matrix algebra simple linear regression algorithm, new x input parameters can be predicted solving the equation 1 applying the obtained θ values. For this propose, a specific IP entity was developed and is presented in Figure 4(b).

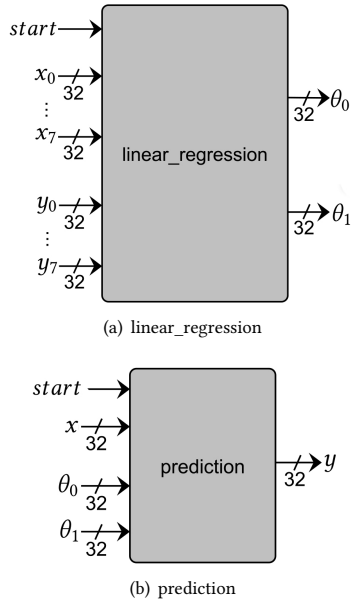


Figure 4: Custom simple linear regression and prediction IP diagrams.

3 SYSTEM EVALUATION

The hardware design was implemented using the Altera Quartus II 12.0 Web Edition. The FPGA physical resource consumption is presented in Table 1. The considerable density of logical elements and of pin number of the FPGA guaranteed its low using percentages. No on-chip memory bits was used, that is, no memory behavior was implemented in the design.

Table 1: FPGA resource consumption by the hardware simple linear regression.

Component	Total	Consumed
Logic elements	114480	8%
Input and output pins	529	1%
On-chip memory bits	3981312	0%
Embedded multipliers	532	89%

The most critical resource was the embedded multipliers, which performed the fixed-point multiplications. Even with the high using percentage, the availability of this component was enough to parallelize the linear_regression and prediction IPs algorithms considering an experimental number of training points. Therefore, each PROCESS of these entities is executed in a single clock cycle. The presented FPGA resource consumption was obtained considering 8 training data points.

Auxiliary entities were declared to support the hardware evaluation. A serial entity managed the communication with the computer. Training data point and hardware prediction results were stored in 32-bit STD_LOGIC_VECTOR SIGNAL VHDL data objects. In prediction phase, it was instanced one prediction entity for each one of

the 8 x training parameters. This custom architecture generated an estimated 136.82 mW thermal power dissipation hardware design.

Two training data sets with 8 points were considered to represent an increasing and a decreasing function. The considerations are presented in sequence.

3.1 First study case: increasing linear regression line.

The 8 training data point applied in this analysis are presented in Table 2. These values are stored in float-point arithmetic in the computer. To be transmitted to the FPGA, the numbers were converted to fixed-point arithmetic. With the hardware linear regression and prediction algorithms finalization, the results are transmitted to the computer and converted back to floating-point notation.

In order to analyze the fixed-point arithmetic operations performed in hardware, a Python script was implemented to solve the equation 2 using the NumPy package functions to perform the matrix algebra operations. The obtained hardware and Python linear regression lines were respectively $y = 3.02501x + 1.41765$ and $y = 3.01922x + 1.41397$, and the predictions of both equations considering the x training values are shown in Table 2.

Table 2: Data of the increasing linear regression study case.

Training data (x, y)	Prediction		Euclidean distance
	Hardware	Python	
(-1.1, -1.7)	-1.90986	-1.90717	0.00269
(0.1, 2.4)	1.72015	1.71589	0.00426
(1.2, 5.0)	5.04766	5.03703	0.01063
(2.3, 7.3)	8.37517	8.35818	0.01699
(3.1, 10.9)	10.79518	10.77355	0.02163
(4.1, 12.5)	13.82019	13.79277	0.02742
(4.8, 16.2)	15.9377	15.90623	0.03147
(5.7, 19.7)	18.66021	18.62352	0.03669

The difference between the hardware fixed-point and floating-point double precision Python algorithms was measured through the calculation of Euclidean distance, which values were inferior to 0.04 for the considered training examples, showing that the proposed fixed-point quantization obtained an acceptable precision. The fixed-point arithmetic precision can be increased considering more bits in the fractional component. However, if the length of the binary word representation does not change, the fixed-point numbers range would decrease since the integer component would have fewer bits.

Figure 5 shows the training data point, training data line and the hardware linear regression line. The Python line was not plotted as the graph resolution was not enough to show the small difference between the hardware and software linear regression predictions.

3.2 Second study case: decreasing line.

To validate the proposed hardware linear regression system in cases with negative slope lines ($\theta_1 < 0$) a specific training data was selected. The same increasing line methodology was applied in the decreasing line case. The training data point, the software

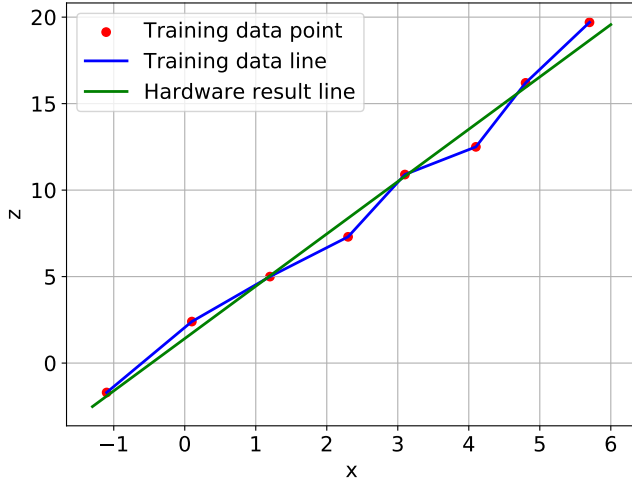


Figure 5: Training points and hardware linear regression increasing line.

and hardware predictions and the Euclidean distance of these sets are presented in Table 3.

Table 3: Data of the decreasing linear regression study case.

Training data (x, y)	Prediction		Euclidean distance
	Hardware	Python	
(-4.0, 8.5)	8.23473	8.25583	0.0211
(-3.3, 7.6)	7.26778	7.2864	0.01862
(-2.2, 5.2)	5.74828	5.76301	0.01473
(-0.7, 3.9)	3.67624	3.68566	0.00942
(0.1, 2.1)	2.57115	2.57774	0.00659
(1.1, 0.8)	1.18979	1.19284	0.00305
(2.3, 0.3)	-0.46784	-0.46904	0.0012
(3.4, -2.1)	-1.98733	-1.99243	0.0051

The computed hardware and software linear regression line were respectively $y = -1.38136x + 2.70929$ and $y = -1.3849x + 2.71623$, which Euclidean distance predictions differences were inferior then 0.02.

Figure 6 shows the training data line and points of the decreasing line study case. As the hardware and software linear regression line were very similar, only the hardware line result was plotted.

With an acceptable precision, the fixed-point arithmetic operations validated the proposed quantization for the hardware linear regression design.

A relevant issue is the high usage rate of the FPGA embedded multipliers. The linear regression algorithm parallelization and the multiple prediction entities instantiations almost consumed the specialized hardware multipliers. This implementation was focused on hardware performance, depriving the FPGA resources optimization consumption. The reduced number of training data point allowed to implement fully parallel entities as presented.

When more complex machine learning algorithms are considered to be implemented within an FPGA, the hardware resources may

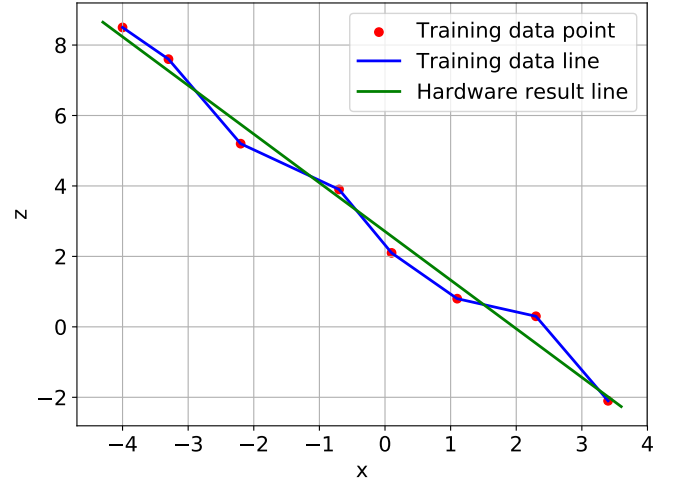


Figure 6: Training points and hardware linear regression decreasing line.

be insufficient due to high data and arithmetic operation density. Thus, it is important to design an architecture considering the algorithm data flow and data storing organization. Efficient memory reading and writing patterns can greatly increase parallelism and data throughput within the FPGA [11].

The results of the presented methodology to solve the linear regression considering a reduced number of training examples had confirmed the correct processing of the matrix algebra operations. As a continuation of this work, it is intended to considerate training sets with more points. To archive this upgrade the algorithm 1 will be adapted to support the processing and storage of more data respecting the hardware available by the FPGA used.

Another parameter that will be studied considering cases with more training data is the fixed-point quantization, which must be correctly modeled to guarantee real-time predictions with adequate precision.

One of the most applied machine learning techniques is the deep learning, which uses multilayered networks to extract attributes from training examples at various levels of abstraction and then performs pattern recognition [20]. Similarity as linear regression, deep learning algorithms are based on matrix algebra operations. Therefore, an important aspect to optimize deep learning implementations in FPGA is to improve matrix algebra operations [18].

4 CONCLUSION

In this paper, a custom fixed-point arithmetic linear regression hardware architecture implemented in the Altera DE2-115 development board was proposed. The hardware design was created using the VHDL hardware description language. Considering a sample 8 training data point, the FPGA resources was enough to implement a custom linear regression and prediction entity which need only one clock cycle to be executed. he power consumption of the proposed embedded FPGA system was estimated to be 136.82 mW.

With the introductory considerations of memory organization and matrix algebra operations, the proposed architecture can be

extended to support training data set with more points, and even integrate more complex machine learning hardware implementations, such as deep learning systems.

ACKNOWLEDGMENTS

This study was financed in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) - Finance Code 001.

REFERENCES

- [1] Juan José Rodríguez Andina, Eduardo De la Torre Arnanz, and Maria Dolores Valdes. 2017. *FPGAs: Fundamentals, Advanced Features, and Applications in Industrial Electronics*. CRC Press.
- [2] D. Chen and S. Ko. 2007. Design and Implementation of Decimal Reciprocal Unit. In *2007 Canadian Conference on Electrical and Computer Engineering*. 1094–1097. <https://doi.org/10.1109/CCECE.2007.279>
- [3] Lattice Semiconductor Corporation. 2009. *DSP Guide for FPGAs*.
- [4] Andreea-Ingrid Cross, Liucheng Guo, Wayne Luk, and Mark Salmon. 2018. CRRS: Custom Regression and Regularisation Solver for Large-Scale Linear Systems. *2018 28th International Conference on Field Programmable Logic and Applications (FPL)* (2018), 389–3894.
- [5] J. Dean, D. Patterson, and C. Young. 2018. A New Golden Age in Computer Architecture: Empowering the Machine-Learning Revolution. *IEEE Micro* 38, 2 (Mar 2018), 21–29. <https://doi.org/10.1109/MM.2018.112130030>
- [6] Pablo Royer del Barrio, Miguel Ángel Sánchez Marcos, Marisa López Vallejo, and Carlos Alberto López Barrio. 2011. Area-Efficient Linear Regression Architecture for Real-Time Signal Processing on FPGAs. In *Proceedings of 26th Conference on Design of Circuits and Integrated Systems*. <http://oa.upm.es/12192/>
- [7] Intel FPGA. 2019. DSP Builder for Intel FPGAs. Retrieved May 31, 2019 from <https://www.intel.la/content/www/xl/es/software/programmable/quartus-prime/dsp-builder.html>
- [8] A. Habegger, A. Stahel, J. Goette, and M. Jacomet. 2010. An Efficient Hardware Implementation for a Reciprocal Unit. In *2010 Fifth IEEE International Symposium on Electronic Design, Test Applications*. 183–187. <https://doi.org/10.1109/DELTA.2010.65>
- [9] K. Kara, D. Alistarh, G. Alonso, O. Mutlu, and C. Zhang. 2017. FPGA-Accelerated Dense Linear Machine Learning: A Precision-Convergence Trade-Off. In *2017 IEEE 25th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. 160–167. <https://doi.org/10.1109/FCCM.2017.39>
- [10] F. Liang, Y. Yang, G. Zhang, X. Zhang, and B. Wu. 2018. Design of 16-bit fixed-point CNN coprocessor based on FPGA. In *2018 IEEE 23rd International Conference on Digital Signal Processing (DSP)*. 1–5. <https://doi.org/10.1109/ICDSP.2018.8631564>
- [11] L. Ma, L. Lavagno, M. T. Lazarescu, and A. Arif. 2017. Acceleration by Inline Cache for Memory-Intensive Algorithms on FPGA via High-Level Synthesis. *IEEE Access* 5 (2017), 18953–18974. <https://doi.org/10.1109/ACCESS.2017.2750923>
- [12] M. Mohammadi, A. Al-Fuqaha, S. Sorour, and M. Guizani. 2018. Deep Learning for IoT Big Data and Streaming Analytics: A Survey. *IEEE Communications Surveys Tutorials* (2018), 1–1. <https://doi.org/10.1109/COMST.2018.2844341>
- [13] R. Fernandez Molanes, K. Amarasinghe, J. Rodriguez-Andina, and M. Manic. 2018. Deep Learning and Reconfigurable Platforms in the Internet of Things: Challenges and Opportunities in Algorithms and Hardware. *IEEE Industrial Electronics Magazine* 12, 2 (June 2018), 36–49. <https://doi.org/10.1109/MIE.2018.2824843>
- [14] D. H. Noronha, P. H. W. Leong, and S. J. E. Wilton. 2018. Kibo: An Open-Source Fixed-Point Tool-kit for Training and Inference in FPGA-Based Deep Learning Networks. In *2018 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. 178–185. <https://doi.org/10.1109/IPDPSW.2018.00034>
- [15] Josh Patterson and Adam Gibson. 2017. *Deep learning: A practitioner's approach*. "O'Reilly Media, Inc."
- [16] J. C. Porcello. 2017. Designing and implementing Machine Learning Algorithms for advanced communications using FPGAs. In *2017 IEEE Aerospace Conference*. 1–10. <https://doi.org/10.1109/AERO.2017.7943637>
- [17] George AF Seber and Alan J Lee. 2012. *Linear regression analysis*. Vol. 329. John Wiley & Sons.
- [18] Hendrik Woehrle and Frank Kirchner. 2018. CAEMO - A Flexible and scalable high performance matrix algebra coprocessor for embedded reconfigurable computing systems. *Microprocessors and Microsystems* 56 (2018), 47 – 63. <https://doi.org/10.1016/j.micpro.2017.10.005>
- [19] Xilinx. 2019. DSP Solutions. Retrieved May 31, 2019 from <https://www.xilinx.com/products/technology/dsp.html>
- [20] Qingchen Zhang, Laurence T. Yang, Zhikui Chen, and Peng Li. 2018. A survey on deep learning for big data. *Information Fusion* 42 (2018), 146 – 157. <https://doi.org/10.1016/j.inffus.2017.10.006>